

卖点：输入解耦连续表示by独立的音频编码器？

FireRedAudio: 一种具有用于理解和生成的解耦连续表示的通用的音频语言模型

FireRed 团队 小红书

摘要

一个统一的音频模型必须能够识别和理解语言、副语言和环境信息，同时支持语音合成与编辑。一个关键挑战在于表示：理解任务偏好紧凑的特征以适应长上下文建模，而语音生成则需要可重建的特征以保留细粒度的声学细节。我们提出了 FireRedAudio，一个具有共享 9B 参数 LLM 的通用的音频语言模型。据我们所知，这是**首个公开披露的统一音频语言模型**，在单个可训练的自回归 LLM 内为理解和生成提供分离的连续输入表示。待识别或分析的音频由专用的音频编码器处理，而用于生成的语音输入则使用基于 RedAE 的路径。LLM 直接生成文本或为流匹配 DiT 提供条件以产生连续的声学隐变量。通过渐进式多任务训练，FireRedAudio 支持 ASR 和音频理解（后者可扩展至长达一小时的录音），以及零样本 TTS、指令 TTS 和语义及声学语音编辑。其对长格式音频的结构化组织实现了秒级时间戳精度。在全面评估中，FireRedAudio 在音频理解 and 多语言 ASR 方面取得有竞争力或领先的性能，在零样本 TTS 中保持较强的内容准确性和说话人保持能力，在指令 TTS 中具有领先的指令遵循能力，并在语义和声学语音编辑上相较于 Ming-UniAudio-Edit 有显著提升。这些结果表明，在中等规模的模型中，**解耦的连续输入表示对于统一音频理解和连续隐变量语音生成是可行的**。我们的代码可在 <https://github.com/FireRedTeam/FireRedAudio> 获取。

1 引言

通用的音频建模涵盖了自动语音识别（ASR）、音频理解、零样本文本语转换合成（TTS）、自然语言控制的 TTS（指令 TTS）、语音编辑以及通过统一接口进行的长时间音频分析。一个共同的语言主干使得这些任务能够复用语言知识、指令遵循能力和推理能力。然而，理解和生成对音频表示提出了不同的要求。理解受益于紧凑的表示，这些表示能够组织任务相关的声学线索以进行长上下文建模，而语音生成则需要可重建的波形表示，以保留音色、韵律、音高和说话风格等细粒度细节。要求一个表示同时提供任务导向的抽象和忠实的波形重建，因此可能会产生性能权衡。先前的工作 [1] 也观察到，**为波形重建而优化的连续自编码器隐变量倾向于强调声学细节，可能不具备高层理解所需的结构，而为识别和分析设计的自监督音频表示通常缺乏直接重建高保真波形的解码接口。****AE**

意思是 VAE （冻结）声学，没有理解（语义）能力？

现有统一音频语言模型中的主流方法是**为生成预测量化的音频 token，同时以不同方式表示输入音频**。LongCat-Next [2] 将输入和输出音频都表示

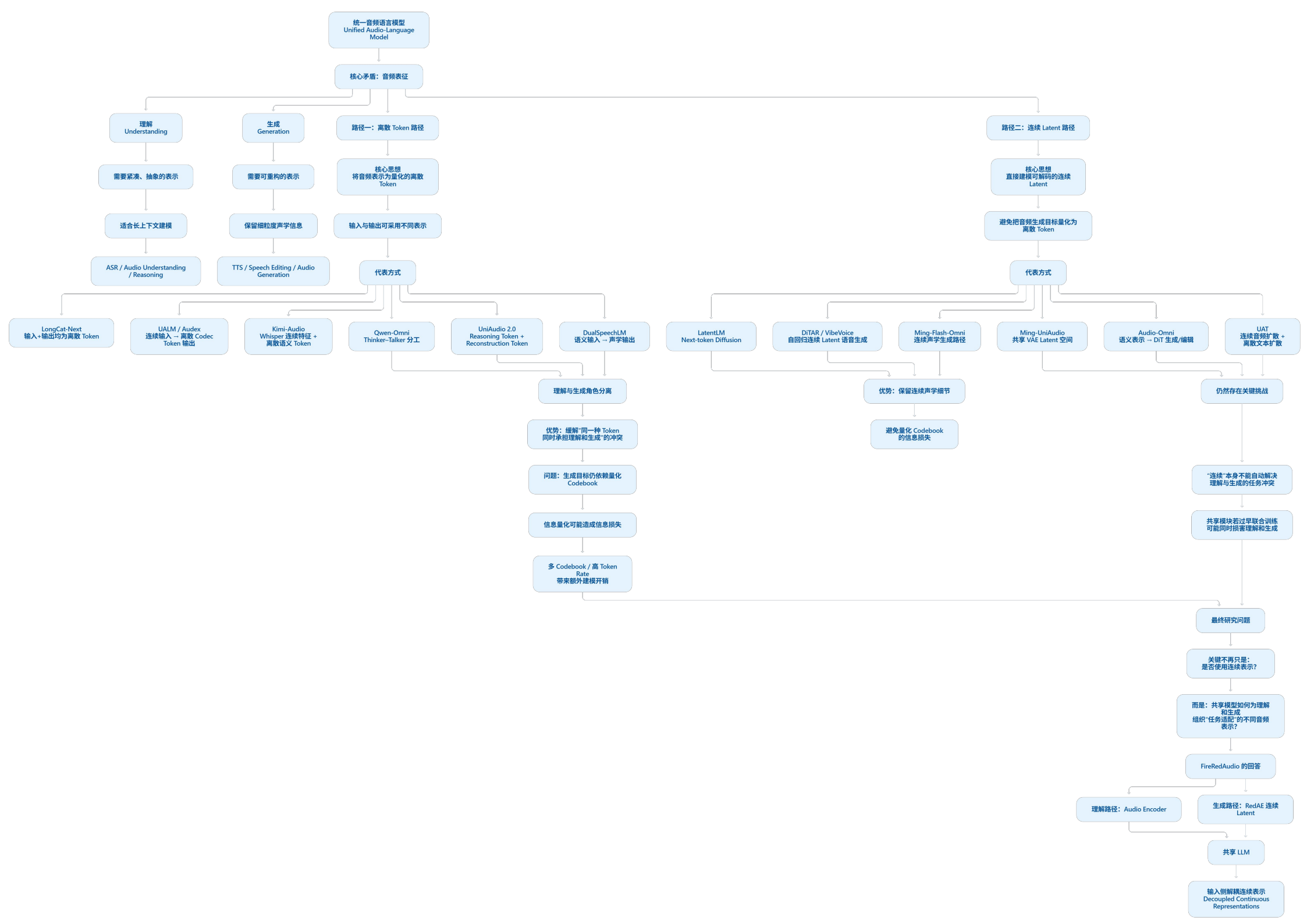
为离散 token。UALM [3] 和 Aude [4] 使用输入音频的连续音频编码器表示，但生成离散编解码器 token。Kimi-Audio [5] 通过元素级加法将连续 Whisper [6] 特征与离散语义 token 的嵌入相结合。Qwen3.5-Omni [7] 和 Qwen3-Omni [8] 则采用 Thinker-Talker 分解：Thinker 处理用于理解和文本生成的连续音频表示，而 Talker 在以 Thinker 提供的特征为条件下预测离散语音 token。UniAudio 2.0 [9] 进一步将音频分解为强调文本对齐语义的推理 token 和保留声学细节的重建 token；其消融结果表明，仅使用重建 token 的理解性能弱于同时使用两种 token 类型。在 DualSpeechLM [10] 中，完整系统采用语义输入和声学输出，在 ASR 和 TTS 性能之间取得了比输入和输出使用相同 token 类型的基线更好的平衡。综合来看，UniAudio 2.0 和 DualSpeechLM 的发现表明，为理解和生成分配不同的表示角色有助于平衡它们的联合性能。尽管如此，这些方法仍依赖量化的、基于码本的目标进行音频生成，这可能引入信息损失；多码本或高 token 率的设计也可能带来额外的建模开销。

另一条研究路径通过直接建模可解码为波形的连续隐变量来避免量化的音频生成目标。LatentLM [11] 引入下一 token 扩散作为连续模态自回归建模的通用接口，而 DiTAR [12] 和 VibeVoice [13] 则证明了自回归连续隐变量建模用于语音合成的可行性。Ming-Flash-Omni [14] 在多模态理解之外，将连续的声学生成路径纳入 Omni 模型。其他方法以不同方式围绕连续表示统一理解和生成。Ming-UniAudio [15] 使用语义模块将由变分自编码器（VAE）[16] 产生的低维隐变量映射到大语言模型（LLM）所消费的表示，使得理解和生成共享植根于同一 VAE 隐空间的表示路径。Audio-Omni [17] 使用冻结的多模态语言模型从文本、音频或视频中提取语义表示，并以这些表示为条件训练一个可生成的扩散 Transformer（DiT）[18] 进行音频生成和编辑，而 UAT [19] 在共享的双流主干中将对连续音频隐变量的扩散与对文本的掩码离散扩散相结合。然而，仅靠连续接口本身并不能自动解决任务冲突。Ming-UniAudio 中的消融研究表明，在早期联合训练期间保持共享语义模块不解冻会损害理解和生成的整体性能。因此，关键问题不仅在于是否使用连续表示，还在于共享模型能否为理解和生成的截然不同的目标组织任务恰当连续输入表示。

受这些观察的启发，我们提出了 FireRedAudio，一个具有用于理解和生成的解耦连续输入表示的统一音频语言模型。待转录或分析的**音频由音频编码器**编码为 12.5Hz 的连续感知表示。**语音生成**路径使用一个预训练的确定性**连续音频自编码器**，称为 RedAE（Red Audio Autoencoder），其隐变量被聚合为 6.25Hz 的 RedAE-Patch 表示。**每个输入音频片段根据其角色被路由到相应的路径**，并与文本指令一起提供给共享的 9B 参数 LLM。对于理解任务，LLM 直接生成文本 token。对于**语音生成任务**，其隐藏状态为一个以流匹配 [20] 训练的 DiT 提供条件，以生成连续的 RedAE 隐变量，随后由 RedAE 解码器渲染为 24kHz 波形。该模型通过文本 token 上的交叉熵和连续隐变量语音生成的流匹配损失进行联合优化。FireRedAudio 支持 ASR 和音频理解（后者可扩展至长达一小时的录音），以及零样本 TTS、指令 TTS 和语音编辑。对于长格式音频的结构化组织，FireRedAudio 展示了秒级时间戳精度。该模型在广泛的评估套件中取得了有竞争力或领先的结果。

我们的主要贡献总结如下：

- **解耦的连续音频输入表示。** FireRedAudio 在一个可训练的自回归 LLM 内引入了两种由独立编码器支持的输入侧连续表示路径。据我们所知，此设计在之前的统一音频语言模型中未见报道。**音频编码器路径服务于理解，而 RedAE-Patch 路径服务于语音生成；两者共享同一语言主干，但不融合其输入表示。**
- **统一的能力与全面的评估。** 我们在 ASR、音频理解（MMAU [21] 和 MMSU [22]）、零样本 TTS（Seed-TTS-Eval [23]）、指令 TTS（InstructTTEval [24]）和语音编辑（Ming-Freeform-Audio-Edit [15]）上评估 FireRedAudio。凭借 9B 参数的 LLM 主干，FireRedAudio 在这些任务中取得了有竞争力或领先的结果。



- **统一的可控语音生成与编辑。** 通过单一连续隐变量语音生成路径，FireRedAudio 支持零样本 TTS、对音色、情感、语速和音量进行自然语言控制的指令 TTS，以及语义内容和声学属性的语音编辑。
- **具有秒级时间戳精度的结构化长格式音频组织。** FireRedAudio 将长达一小时的录音组织为带时间戳的条目，并在 5 至 50 分钟的录音上，在 1 秒边界容差下展示了稳定的时间-内容对齐。

2 方法

2.1 模型概览

图 1: FireRedAudio 概览。用于理解和生成的解耦连续路径对输入进行编码。共享的 LLM 生成文本或为 DiT 提供条件以生成 RedAE 隐变量，这些隐变量被解码为波形。嵌入图说明了每个音频步骤的 DiT 条件。

理解和生成都依赖于语言、副语言和非语言的声学信息，但需要它们处于不同的抽象层次。理解受益于紧凑的表示，这些表示能够组织任务相关的线索以进行识别和分析，而语音生成则额外需要可重建波形的表示，以保留细粒度的声学细节。因此，FireRedAudio 保留了一个共享的 LLM 主干，同时引入了两个解耦的连续输入路径，其表示由独立的编码器产生且不进行融合。如图 1 所示，主模型包括一个音频编码器、一个音频适配器、一个 RedAE 编码器、一个 Patch 编码器、一个共享的 9B 参数 LLM 主干和一个流匹配 DiT。在推理期间，单独加载的 RedAE 解码器将生成的 RedAE 隐变量渲染为波形。

输入路径根据当前任务中每个音频片段的功能角色进行选择。待转录或分析的音频由音频编码器和音频适配器处理为 12.5Hz 的连续感知表示，而用于语音生成任务的语音输入则由 RedAE 编码器编码为 25Hz 的 RedAE 隐变量帧序列。我们将一个音频步骤定义为四个连续的 RedAE 隐变量帧，对应 160ms 的音频。Patch 编码器将这四帧聚合为一个 RedAE-Patch 表示，产生 6.25Hz 的 LLM 侧帧率。来自两条路径的表示既不加法也不融合；相反，它们被插入到对应音频段的位置，并与文本指令一起提供给共享的 LLM。对于理解任务，LLM 自回归地生成文本响应。对于语音生成任务，LLM 每次一个音频步骤地向 DiT 提供条件，DiT 每步生成四个 RedAE 隐变量帧。新生成的帧由 Patch 编码器转换为 RedAE-Patch 表示，并作为下一步的上下文反馈给 LLM。一旦语音生成终止，完整的 RedAE 隐变量序列由 RedAE 解码器渲染为 24kHz 波形。

2.2 解耦的连续音频表示

2.2.1 用于理解的音频编码器

面向理解的表示必须保留语言内容、副语言线索和任务相关的声学证据，同时保持足够的紧凑性以控制长格式音频的序列长度。因此，理解路径接收 16kHz 的单声道波形并提取 log-Mel 特征。音频编码器（包括其卷积前端和 Transformer）从 Whisper-large-v3 编码器 [6] 初始化，从而利用预训练的多语言语音表示和声学-语言对齐先验。一个独立的音频适配器执行额外的时间压缩，并将生成的音频特征映射到 LLM 输入空间。

为控制长格式音频编码的计算成本，100Hz 的 log-Mel 序列首先被划分为最多 30 秒的不重叠窗口。在每个窗口内，音频编码器的卷积前端将特征编码并下采样至 50Hz，之后 Transformer 独立执行局部编码。窗口级输出按其原始时间顺序拼接，并传递给音频适配器，后者进一步将序列下采样至 12.5Hz 并将其投影到 LLM 输入嵌入空间。跨窗口的长程交互随后由共享的 LLM 建模。将额外的时间压缩和跨模态对齐隔离在音频适配器中，也使得模型能够在进一步优化音频编码器之前首先学习该对齐接口。

2.2.2 用于连续隐变量语音生成的 RedAE 和 Patch 编码器

为获得用于语音生成的可重建波形的连续表示，我们独立预训练了 RedAE，一个确定性的音频自编码器，它将波形直接映射到连续隐变量，而不进行变分后验建模或 Kullback-Leibler (KL) 正则化 [25]。仅靠重建训练可能使隐空间被低层声学变化主导 [1]，使自回归建模复杂化并可能加剧误差累积。因此，我们使用蒸馏损失将 RedAE 隐变量与来自冻结教师音频编码器（在语音、通用音频和音乐上训练的理解任务）的高层特征对齐。这种监督鼓励隐变量中的高层结构，促进其随后与 LLM 的对齐，同时保持波形可重建性。与 Ming-UniAudio [15] 在可重建隐变量表示之后附加一个独立的语义模块不同，RedAE 在自编码器预训练期间就纳入了高层音频监督；教师音频编码器随后被丢弃，不增加运行时分支。

如图 2 所示，RedAE 编码器采用因果架构。一个 24kHz 的波形被划分为 480 个样本的不重叠块，形成 50Hz 的帧序列。两个级联的 Qwen3 [26] Transformer 模块分别执行上下文编码和时间聚合。第二个模块聚合每对相邻帧，输出投影产生 25Hz 的 64 维 RedAE 隐变量。RedAE 解码器通过线性投影和通道到时间的重排逆转该时间压缩，随后是一个因果 Qwen3 Transformer 和一个用于波形重建的逆短时傅里叶变换 (iSTFT) 头 [27]。RedAE 在一个包含 50% 清晰语音、25% 带噪语音、10% 音效和 15% 音乐的混合语料上进行预训练。该混合使得 RedAE 能够学习跨语音、环境声音和音乐的可重建连续表示。

图 2: RedAE 的架构与预训练。冻结的教师音频编码器提供高层监督，而可训练的自编码器将 50 Hz 音频帧压缩为 25 Hz 隐变量，并通过 iSTFT 头重建波形。雪花和火焰分别表示冻结和可训练的模块。

遵循 X-Codec 框架 [28]，RedAE 使用混合 GAN 目标进行优化。由于 RedAE 不对隐后验进行参数化，生成器目标省略了 KL 正则化，而是纳入了语义蒸馏：

$$\mathcal{L}_G = \lambda_{\text{rec}} \mathcal{L}_{\text{rec}} + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}} + \lambda_{\text{fm}} \mathcal{L}_{\text{fm}} + \lambda_{\text{distill}} \mathcal{L}_{\text{distill}}.$$

这里， $\mathcal{L}_{\text{rec}}$ 、 $\mathcal{L}_{\text{adv}}$ 和 $\mathcal{L}_{\text{fm}}$ 分别表示重建损失、对抗损失和判别器特征匹配损失。 $\mathcal{L}_{\text{distill}}$ 是一个均方误差 (MSE) 语义蒸馏损失，将 RedAE 隐变量与冻结教师音频编码器的特征对齐。该目标仅用于独立的 RedAE 预训练。在 FireRedAudio 训练期间，预训练的 RedAE 编码器被冻结，RedAE 解码器不加载；解码器仅用于推理。

将原生的 RedAE 隐变量帧逐个输入 LLM 将大幅增加音频序列长度和自回归建模成本。因此，我们仅在 LLM 接口处引入一个 Patch 编码器。一个局部 Transformer 将每四个连续的 25Hz RedAE 隐变量帧聚合为一个与 LLM 输入嵌入维度相同的 RedAE-Patch 表示，将 LLM 侧音频帧率降至 6.25Hz。波形重建和基于 DiT 的语音生成仍保持在原生的 RedAE 隐空间中进行。

2.3 使用共享 LLM 的统一自回归建模

FireRedAudio 支持具有异构输入-输出结构的任务：ASR 和音频理解将音频映射到文本；零样本 TTS 使用参考语音从文本合成目标语音；指令 TTS 遵循自然语言指令控制语音生成；语音编辑根据编辑指令变换源波形。为在一个模型内统一这些任务，我们采用 Qwen3.5-9B¹ 作为共享语言主干，并将每个任务表述为一个序列到序列的问题，其中输入和输出 token 序列通过任务特定的模板进行组织。音频输入通过两条解耦路径之一表示。

对于理解任务，输入序列由系统提示、任务指令、音频路径 token 和处理后的音频表示组成，随后是助理前缀。LLM 以标准的自回归方式生成文本响应。音频表示由音频编码器-适配器路径以 12.5Hz 产生；生成过程中不涉及 DiT。

对于语音生成任务，输入序列包含系统提示、任务指令、描述生成语音特征的文本 token（如说话人参考的风格或所需的韵律属性），以及由 Patch 编码器以 6.25Hz 产生的 RedAE-Patch 音频上下文。每个 RedAE-Patch 表示在 LLM 中被视为一个特殊的音频步骤 token，用于触发连续语音生成，而不是一个被预测的离散文本 token。当 LLM 生成一个音频步骤 token 时，它不产生文本响应，而是将其当前隐藏状态提供给 DiT 以生成四个 RedAE 隐变量帧。然后，Patch 编码器将生成的帧转换回 RedAE-Patch 表示，并附加到输入序列中作为后续步骤的音频上下文。当 LLM 最终预测音频结束 token 时，音频步骤生成终止。该设计使得自回归 LLM 能够通过预测文本 token 和控制语音结构的特殊 token 来建模高层序列结构，而将细粒度的连续声学生成委托给 DiT。

2.4 使用流匹配 DiT 的连续隐变量语音生成

对于每个音频步骤，共享的 LLM 产生一个隐藏状态，该状态总结了任务指令和所有前面的序列上下文。DiT [18] 使用包含当前和前两个音频步骤的隐藏状态的局部窗口，来生成当前步骤的四个 RedAE 隐变量帧。这种划分使得 LLM 能够建模序列级上下文，而 DiT 处理细粒度的连续声学。我们使用流匹配 [20] 训练 DiT，以学习从高斯噪声到目标 RedAE 隐变量的条件速度场，从而避免量化的语音 token。

在训练期间，冻结的 RedAE 编码器将每个目标波形转换为真实的 RedAE 隐变量，这些隐变量被分组为每四个连续帧的音频步骤。对于每个目标步骤，我们采样高斯噪声和流匹配时间 t ，在噪声和四个目标帧之间进行插值以获得带噪输入，并使用相应概率路径的速度作为回归目标。除了带噪的当前步骤隐变量和 t 之外，DiT 还接收两个互补的条件信号。

声学条件包括来自前两个音频步骤的八个历史 RedAE 隐变量帧，与当前步骤的四个带噪帧拼接，形成一个 12 帧的声学输入。在训练期间，真实的 RedAE 隐变量提供声学历史。LLM 条件包括紧接在当前步骤真实 RedAE-Patch 之前的因果隐藏状态，以及前两个步骤对应的隐藏状态。由于当前状态是在 LLM 消费当前真实补丁之前获得的，目标不会泄漏到其自身的条件中。三个隐藏状态按时间顺序排列，并各自重复四次以与 RedAE 隐变量帧对齐。任一条件窗口中缺失的历史用零填充。LLM 条件捕获指令和因果序列上下文，而声学条件促进局部连续性。

时间对齐的 LLM 条件沿特征维度与 12 帧声学输入拼接，而 t 通过单独的时间嵌入注入。仅 DiT 在最后四个位置的输出贡献损失；前八个位置提供声学上下文。设 $\mathcal{P}$ 表示训练数据中有效目标音频步骤的集合， $\hat{V}_j$ 和 V_j^* 分别表示第 j 个音频步骤的预测速度和目标速度。流匹配损失为

$$\mathcal{L}_{\text{flow}} = \frac{1}{|\mathcal{P}|} \sum_{j \in \mathcal{P}} \text{MSE}(\hat{V}_j, V_j^*).$$

MSE 在目标音频步骤的四个帧和 64 个特征维度上取平均。在训练期间，我们随机丢弃 LLM 条件以启用无分类器引导 [29]，同时始终保留声学条件。

在推理时，语言模型头预测的每个音频步骤 token 调用一个 DiT 步骤。两步骤的声学历史缓冲区用零样本 TTS 中参考语音的最后两步初始化，其他语音生成任务则用零初始化。用于语音编辑的源音频仅通过 LLM 条件影响 DiT，不插入此缓冲区。从高斯噪声开始，DiT 积分学习到的速度场以生成四个 RedAE 隐变量帧。这些帧更新声学历史缓冲区（保留最近两个步骤），并由 Patch 编码器转换为 RedAE-Patch 表示返回给 LLM。生成持续进行，直到语言模型头预测音频结束 token，之后 RedAE 解码器从完整的隐变量序列重建 24kHz 波形。

2.5 联合训练目标

在 RedAE 编码器冻结的情况下，FireRedAudio 联合优化自回归 token 目标和连续流匹配目标：

$$\mathcal{L} = \lambda_{\text{text}} \mathcal{L}_{\text{text}} + \lambda_{\text{flow}} \mathcal{L}_{\text{flow}}.$$

每个损失在其自身的有效位置上归一化，我们设置 $\lambda_{\text{text}} = \lambda_{\text{flow}} = 1$ 。由于在混合训练语料中具有连续声学目标的示例采样频率较低，等权重避免了进一步削弱连续声学学习信号。

$\mathcal{L}_{\text{text}}$ 对普通文本以及音频开始、音频步骤和音频结束 token 应用下一 token 交叉熵，同时排除提示、填充和条件音频。普通文本和推理 token，连同音频开始和音频结束 token，使用单位权重。由于音频步骤位置在长语音序列中数量众多，我们为其分配权重 $w_a = 0.01$ 。这些特殊 token 仅控制输出语音序列的结构；其连续的声学内容由 DiT 生成。

设 $\mathcal{T}$ 和 $\mathcal{A}$ 分别表示单位权重和音频步骤位置， $\ell_i = -\log p(y_i | y_{<i}, \text{context})$ 。我们计算

$$\mathcal{L}_{\text{text}} = \frac{\sum_{i \in \mathcal{T}} \ell_i + w_a \sum_{i \in \mathcal{A}} \ell_i}{|\mathcal{T}| + w_a |\mathcal{A}|}, \quad w_a = 0.01.$$

因此， w_a 同时加权分子和有效 token 分母，而非将最终损失作为一个整体缩放。分母跨数据并行 worker 聚合以获得全局加权 token 平均。联合目标更新共享的 LLM、音频编码器、音频适配器、Patch 编码器和 DiT。

3 训练策略

FireRedAudio 结合了预训练模块和新初始化的跨模态及语音生成组件。音频编码器、LLM 和 RedAE 是预训练的，而音频适配器必须建立理解接口，Patch 编码器和 DiT 必须与共享 LLM 建立连续的隐变量语音生成接口。从最开始联合优化所有可训练组件将使 LLM 暴露于不稳定的理解和语音生成接口。因此，我们采用五阶段渐进策略。适配器对齐首先稳定理解路径，然后音频编码器适应拓宽它。统一中期训练联合优化所有可训练模块并引入连续隐变量语音生成目标。多任务后训练添加指令 TTS 和语音编辑，同时加强细粒度、结构化和多步骤音频理解。最后，长上下文扩展将音频理解扩展到长达一小时的录音。RedAE 编码器在整个主模型训练期间保持冻结。

3.1 数据混合与采样

统一多任务训练所需的数据量差异很大。纯文本语言建模、ASR 和零样本 TTS 数据比指令 TTS、语音编辑和细粒度音频理解的数据丰富得多；在每个任务家族内部，跨语言、声学领域和数据源也存在进一步的不平衡。因此，按原始数据规模比例采样将使大型任务和数据集主导优化。Conneau 等人 [30] 在多语言语音预训练中对语言的采样分布应用幂律平滑。我们将此思想扩展到两级层次结构：首先采样一个任务家族，然后采样其组成数据集。

设 W_i 表示第 i 个任务家族的配置采样权重， w_{ij} 表示其第 j 个组成数据集的权重。两级采样概率为

$$p(j \mid i) = \frac{W_i^{\alpha_{\text{task}}}}{\sum_k W_k^{\alpha_{\text{task}}}}, \quad p(j \mid i) = \frac{w_{ij}^{\alpha_{\text{data}}}}{\sum_l w_{il}^{\alpha_{\text{data}}}}.$$

我们分别设置任务级和数据集级的平滑指数为 $\alpha_{\text{task}} = 0.5$ 和 $\alpha_{\text{data}} = 0.7$ 。较小的任务级指数提供跨任务家族的更强平衡，而数据集级指数缓和同一家族内数据集之间的规模差异。我们根据各阶段的目标调整跨训练阶段的任务家族权重，从而控制优化期间每个任务的采样频率。

我们使用术语“多模态 token”来统称进入 LLM 的文本 token 或连续音频表示。每个阶段的训练量由优化期间处理的无填充多模态 token 总数衡量，以十亿 token（B token）报告。表 1 总结了每个阶段的数据焦点、训练量和每示例最大序列长度。

表 1：五阶段训练策略概览。训练量以十亿无填充多模态 token 衡量，四舍五入到最接近的十亿。序列长度表示每示例的最大有效长度。

阶段	数据焦点	训练量	序列长度
适配器对齐	中英文 ASR；多语言 ASR	180B	8k
音频编码器适应	语音、通用音频和音乐的 ASR 与理解	390B	8k
统一中期训练	文本、ASR、音频理解、零样本 TTS 和音频-文本交错	990B	8k
多任务后训练	指令 TTS 和语音编辑；保留 ASR、音频理解、纯文本指令数据和零样本 TTS	511B	8k
长上下文扩展	长达一小时录音的长格式音频理解；保留常规长度多任务数据	591B	200k

3.2 渐进式训练

3.2.1 适配器对齐

尽管音频编码器提供了预训练的语音表示，LLM 也具备预训练的语言建模能力，但随机初始化的音频适配器还无法可靠地连接它们。在此接口对齐之前更新两个预训练组件可能会使跨模态映射中的噪声干扰它们已有的表示。因此，我们使用 80B 中英文 ASR token 和 100B 多语言 ASR token，因为转录提供了密集且定义良好的音频-文本监督。音频编码器和 LLM 被冻结，仅使用自回归文本交叉熵优化音频适配器。由于适配器在此阶段是随机初始化并单独训练的，其学习率线性预热 500 步至 2×10^{-4} ，然后保持不变。此阶段教导音频适配器对音频编码器特征进行时间压缩并将其映射到 LLM 输入空间，在音频前端进一步优化之前建立一个稳定的接口。

3.2.2 音频编码器适应

由于第一阶段使用的转录监督主要捕获语言内容，第二阶段将训练信号拓宽至覆盖副语言和说话人信息、环境声音、音乐和时间关系。除了 120B 中英文 ASR token 和 120B 多语言 ASR token 外，此阶段还使用 150B 音频理解 token 将感知范围扩展到转录之外。一旦跨模态接口稳定，我们解冻音频编码器并使用自回归文本交叉熵将其与音频适配器联合优化，同时保持 LLM 冻结作为固定的语言主干。我们为音频编码器和适配器保留 2×10^{-4} 的峰值学习率，经过 2.5k 步线性预热后线性衰减。

3.2.3 统一中期训练

在理解路径对齐后，我们将课程扩展到语言建模、音频理解和零样本 TTS。在此阶段，我们首次联合优化音频编码器、音频适配器、LLM、Patch 编码器和 DiT。混合包含 990B 多模态 token：160B 中英文 ASR token、200B 多语言 ASR token、250B 音频理解 token、200B 纯文本 token、120B 零样本 TTS token 和 60B 音频-文本交错 token。纯文本数据保留语言建模和指令遵循。在交错示例中，前面的音频由理解路径编码为 LLM 上下文，仅预测后续文本，从而加强音频-语言对齐。零样本 TTS 数据引入连续隐变量语音生成目标。

我们不单独预训练 Patch 编码器或 DiT。Patch 编码器学习将 RedAE 隐变量映射为共享 LLM 可作为音频上下文消费的 RedAE-Patch 表示，而 DiT 学习在以相应 LLM 隐藏状态为条件下生成连续 RedAE 隐变量。因此，它们通过完整的连续隐变量语音生成循环联合训练。在 RedAE 编码器冻结以保持静止隐空间的情况下，所有可训练模块使用第 2.5 节定义的加权文本交叉熵和流匹配损失联合优化。前者监督文本输出和音频率序列控制 token，而后者监督连续声学内容。由于预训练的 LLM 在理解和生成之间共享，我们保守地更新它。同时，音频编码器和音频适配器继续适应统一的训练分布，而 Patch 编码器和 DiT 建立连续隐变量语音生成循环。因此，我们对 LLM 使用比音频模块更小的峰值学习率。具体来说，我们在 2.5k 步内将学习率线性预热至 LLM 主干 3×10^{-5} 和四个音频模块 2×10^{-4} 的峰值，随后线性衰减。此阶段的目标是为语言建模、音频感知和连续隐变量语音生成建立共享基础，而非为个别任务专门化模型。

3.2.4 多任务后训练

中期训练从大规模数据中建立广泛基础，而第四阶段强调指令遵循。为保留先前获得的能力，此阶段使用 13B 高质量中英文 ASR token、4B 多语言 ASR token、300B 音频理解 token、60B 纯文本指令 token 和 100B 零样本 TTS token。它还额外引入 22B 指令 TTS token 和 12B 语音编辑 token，总计约 511B token。在指令 TTS 和语音编辑数据以及需要多步推理的音频理解示例中添加显式的思维链（CoT）监督。这种监督鼓励模型在生成目标语音或文本答案之前，对合成条件、编辑指令或音频证据进行推理。

后训练用纯文本指令数据替换非指令性纯文本数据，同时保留 ASR、零样本 TTS 和音频理解监督以减轻遗忘。可训练模块集合和联合目标与中期训练保持不变。由于指令 TTS 和语音编辑要求共享 LLM 学习新的指令条件输出模式，我们为 LLM 主干保留 3×10^{-5} 的峰值学习率。到本阶段，音频编码器-适配器理解路径和 Patch 编码器-DiT 连续隐变量语音生成路径均已建立。因此，我们将所有四个音频模块的峰值学习率降至 1×10^{-4} 以进行任务特定细化。经过 500 步预热后，两个学习率水平线性衰减。

3.2.5 长上下文扩展

在长上下文扩展之前，音频理解示例最多限制为 5 分钟，不足以整合长格式录音中远距离时间区域上的证据。在最后阶段，我们将统一文本-音频率序列的每示例最大有效长度从 8k 增加到 200k，并将理解的最大音频时长延长至一小时。

3.3 优化配置

表 2 报告了 LLM 主干、音频编码器、音频适配器、Patch 编码器和 DiT 的阶段式学习率，以及学习率调度、冻结模块和每副本打包预算。为减少可变长度多模态示例的填充，我们在阶段特定的多模态 token 计数限制和理解与语音生成路径的独立音频时长预算下构建每个打包的微批次。这些约束使得跨异构多模态批次的每副本内存使用保持稳定。

表 2：五个训练阶段的优化配置。破折号表示该模块在该阶段未被优化。

超参数	理解对齐	统一多任务训练	长上下文扩展	:-- :-- :-- :-- :-- :--	适配器对齐	编码器适应	统一中期训练	多任务后训练	扩展																																										
峰值 LR (LLM 主干)	冻结	冻结	3×10^{-5}	3×10^{-5}	1×10^{-5}	峰值 LR (音频编码器)	冻结	2×10^{-4}	2×10^{-4}	1×10^{-4}	3×10^{-5}	峰值 LR (音频适配器)	2×10^{-4}	2×10^{-4}	2×10^{-4}	1×10^{-4}	3×10^{-5}	峰值 LR (Patch 编码器和 DiT)	-- --	2×10^{-4}	1×10^{-4}	3×10^{-5}	LR 调度	常数	线性	线性	线性	线性	预热步数	500	2.5k	2.5k	500	500	每副本最大打包 token 数	36k	36k	36k	36k	200k	每副本最大理解音频时长 (秒)	1,500	1,500	1,000	1,000	3,600	每副本最大语音生成时长 (秒)	-- --	610	610	610

4 实验

4.1 ASR 与音频理解

我们在 MMAU 和 MMSU 上评估音频理解。MMAU [21] 包含 10,000 个多项选择题，涵盖对语音、环境声音和音乐的信息提取和推理；我们报告其 1,000 题的 test-mini 和 9,000 题的 test 划分。MMSU [22] 包含 5,000 个问题，涵盖 47 个感知和推理任务，评估细粒度的英语语音信息，包括语言内容、音系、说话人特征和说话风格。这两个基准共同评估通用音频理解的广度和详细的口语理解。两者均报告准确率（ACC，%），越高越好。

如表 3 所示，FireRedAudio 在所有三个设置中均达到最高准确率。在 MMAU 和 MMSU 上的持续优势表明其对通用音频理解和细粒度口语理解均有强覆盖。这些结果进一步表明，专用的音频编码器路径在无需将其表示调整为语音生成的重建导向要求的情况下，保持了强大的理解性能。

表 3：MMAU 和 MMSU 上的音频理解准确率（%）。越高越好。最佳结果以粗体显示。标有 * 的结果来自我们自己的评估。

模型	MMAU test-mini	MMAU test	MMSU
Step-Audio-R1.1 [31]	77.7	-	75.9
Step-Audio 2 [32]	78.0	-	-
MiMo-Audio-7B-Instruct [33]	74.9	-	61.7
Kimi-Audio [5]	65.2	-	-
LongCat-Next [2]	76.4	-	-
Qwen3-Omni-30B-A3B-Instruct [8]	77.5	-	69.0
Gemini 3.1 Pro [34]	80.7*	78.8*	82.7*

模型	MMAU test-mini	MMAU test	MMSU
Qwen3.5-Omni-Plus [7]	81.4*	79.9*	80.7*
FireRedAudio	82.0	80.9	83.3

我们的 ASR 套件涵盖普通话、英语、多语言、方言、网络、会议和歌唱语音。它包括 AISHELL-1 [35]、AISHELL-2 test-ios [36]、WenetSpeech Test_Net/Test_Meeting [37]、LibriSpeech test-clean/test-other [38]、KeSpeech [39] 和 Opencpop [40]。对于 FLEURS [41]，我们报告英语、中文以及所有 102 种语言的宏平均。根据需要报告 CER 或 WER，越低越好。

表 4 显示 FireRedAudio 在 LibriSpeech test-clean、FLEURS English 和 FLEURS-102 宏平均上达到了最低报告错误率，后者达到 14.94%。它还在 WenetSpeech Test_Meeting 上排名第二，并在 AISHELL-1、KeSpeech 和 Opencpop 上位列前三报告系统。总体而言，这些结果表明在一个同时支持音频理解和语音生成的统一模型中，跨语言和声学条件具有广泛的 ASR 能力。

表 4：跨语音、多语言和歌唱语音基准的 ASR 结果。根据情况报告 CER 或 WER，越低越好。标有 * 的结果来自我们自己的评估。

模型	AISHELL-1	AISHELL-2 test-ios	WenetSpeech Test_Net	Test_Meeting	LibriSpeech test-clean/test-other	FLEURS en/zh	FLEURS-102* 平均	KeSpeech	Opencpop
Step-Audio 2 [32]	0.63	2.10	4.67	14.75	1.17/12.42	3.03/12.68	-	3.63	-
MiMo-Audio-7B-Instruct [33]	1.65	-	3.50	1.	-	-	-	-	-
Ming-UniAudio-16B-A3B [15]	-	2.84	-	1.62	1.	-	-	-	-
Kimi-Audio [5]	0.60	2.56	5.37	16.28	1.28/12.42	4.44/12.69	-	-	-
LongCat-Next [2]	1.47	2.82	5.98	18.19	1.63/13.42	5.24/13.24	-	-	-
Qwen3-Omni-30B-A3B-Instruct [8]	-	-	4.69	15.89	1.22/12.48	2.72/12.20	-	-	1.54
Gemini 3.1 Pro [34]	3.66*	7.10*	11.53	114.21	3.36/14.41	2.97*/14.28*	18.23*	23.67	6.83
Qwen3.5-Omni-Plus [7]	0.82*	2.26*	4.30	15.84	1.11/12.23	3.33*/12.46*	23.66*	3.46	1.49
FireRedAudio	0.71	2.63	5.18	15.33	0.67/12.91	2.53/13.14	14.94	4.82	1.63

4.2 零样本 TTS

我们在 Seed-TTS-Eval [23] 上评估零样本 TTS，其中包含来自 Common Voice [48] 的 1,000 个英语样本和来自 DiDiSpeech-2 [49] 的 2,000 个中文样本。每个示例将参考语音与目标文本配对。对于 FireRedAudio，我们使用 Paraformer-zh5 [50] 计算中文 CER，使用 Whisper-large-v3 [6] 计算英文 WER，使用 WavLM-large [51] 计算说话人相似度（SIM）。较低 CER/WER 和较高的 SIM 更好；平均列总结了中文和英文分数。

如表 5 报告，FireRedAudio 在 Seed-ZH 上达到最佳内容准确率，在 Seed-EN 上排名第二，在所有对比系统中取得了最低的平均内容错误率 1.20%。其平均 SIM 为 0.71，是所有报告双语说话人相似度的统一模型中最高的。尽管其说话人相似度仍低于最强的专用 TTS 系统，但 FireRedAudio 不使用专用说话人嵌入，而是直接以参考语音的 RedAE 表示为条件进行合成。这些结果表明，同时支持音频理解和语音生成并不妨碍目标文本的准确合成或参考说话人线索的保持。强内容准确率也与架构分工一致：共享 LLM 建模目标文本和生成上下文，而 DiT-RedAE 路径生成对应的连续声学。

表 5：Seed-TTS-Eval 上的零样本 TTS 结果。较低 CER 和 WER 及较高的 SIM 表示更好的性能。平均列报告中文和英文分数的均值。每列中最佳和次佳结果分别以粗体和下划线显示。

模型	Seed-ZH	Seed-EN	平均
	CER (%) ↓	SIM ↑	WER (%) ↓

模型	Seed-ZH	Seed-EN	平均
专用 TTS 模型			
Seed-TTS [23]	1.12	0.80	2.25
FireRedTTS [42]	1.51	0.65	3.82
FireRedTTS-2 [43]	1.14	0.74	1.95
DiTAR-1B [12]	1.02	0.75	1.69
F5-TTS [44]	1.56	0.74	1.83
CosyVoice 2 [45]	1.45	0.75	2.57
CosyVoice 3-1.5B [46]	1.12	0.78	2.21
统一音频模型			
MiMo-Audio-7B-Instruct [33]	1.96	-	5.37
Qwen2.5-Omni-7B (RL) [47]	1.42	0.75	2.33
Qwen3-Omni-30B-A3B-Instruct [8]	1.07	-	1.39
Ming-UniAudio-16B-A3B [15]	0.95	0.70	1.85
FireRedAudio	0.83	0.74	1.56

4.3 指令 TTS

表 6: InstructTTSEval 上的指令遵循准确率 (%)。APS、DSD 和 RP 分别评估结构化属性控制、自由形式风格描述遵循和角色/场景推理。VD 表示 Voice Design。所有指标越高越好。最佳结果以粗体显示。所有结果均来自我们使用 Gemini 2.5 Pro 的重新评估。

模型 中文 英文 :--- :--- :--- :--- :--- :--- :--- :--- APS ↑ DSD ↑ RP ↑ APS ↑ DSD ↑ RP ↑ VoiceSculptor-VD [52] 74.6 63.5 62.0 - - - MOSS-VoiceGenerator [53] 71.6 72.5 61.3 58.8 71.8 61.6 Ming-Omni-TTS-16B-A3B [54] 84.6 70.7 56.0 - - - Qwen3-TTS-VD [55] 83.7 81.7 65.8 76.4 81.4 64.2 FireRedAudio 86.0 84.1 70.1 81.1 83.6 70.3
--

我们在 InstructTTSEval [24] 上评估指令 TTS，其中包含总计 6,000 个中英文示例。其三个任务抽象程度递增：声学参数规范（APS）给出结构化的声音属性，描述性风格指令（DSD）使用自由形式的风格描述，角色扮演（RP）需要从角色或场景中推断出合适的声音特征。由于基准的原始 Gemini 2.5 Pro Preview 评估器不再可用，我们使用官方提示以 Gemini 2.5 Pro [56] 重新评估所有系统。我们报告指令遵循准确率，越高越好。

如表 6 所示，FireRedAudio 在所有六个中英文设置中均达到最高准确率。这些增益跨越了结构化属性控制、自由形式风格描述和角色/场景推理，展示了在不同抽象层次上一致的指令遵循。最大的差距是英文 RP 上的 6.1 个百分点，表明在从上下文线索推断未说明的说话风格方面具有特别的优势。这种行为可能受益于后训练期间使用的显式 CoT 监督，它鼓励共享 LLM 在为 DiT 声学生成器提供条件之前推断合适的说话风格。

4.4 语音编辑

表 7: Ming-Freeform-Audio-Edit 上的语义编辑结果。较低的 WER/no-edit WER 和较高的 SIM/ACC 表示更好的性能。最佳结果以粗体显示。

任务	设置	指标	Ming-UniAudio-Edit [15] (中文 英文)	FireRedAudio (中文 英文)
删除	basic	WER (%) ↓	11.89	14.85 10.82
		SIM ↑	0.78	0.76 0.79
		ACC (%) ↑	100.00	82.22 100.00
	open	no-edit WER (%) ↓	11.49	24.26 10.70
		WER (%) ↓	22.92	27.60 10.49
		SIM ↑	0.81	0.74 0.80
插入	basic	ACC (%) ↑	82.92	85.00 89.32
		no-edit WER (%) ↓	17.50	35.21 7.84
		WER (%) ↓	3.42	6.63 3.28
	open	SIM ↑	0.83	0.79 0.83
		ACC (%) ↑	80.00	71.43 83.53

任务	设置	指标	Ming-UniAudio-Edit [15] (中文	英文)	FireRedAudio (中文	英文)
替换	basic	no-edit WER (%) ↓	3.52	17.70	3.51	16.56
		WER (%) ↓	3.89	7.59	2.57	6.98
		SIM ↑	0.83	0.79	0.83	0.84
	open	ACC (%) ↑	79.31	62.31	86.90	69.85
		no-edit WER (%) ↓	4.10	18.84	2.77	17.83
		WER (%) ↓	4.52	8.99	2.66	4.46
删除	basic	SIM ↑	0.82	0.78	0.84	0.81
		ACC (%) ↑	78.62	59.78	87.42	75.98
		no-edit WER (%) ↓	4.63	19.28	2.91	16.34
	open	WER (%) ↓	4.56	7.64	2.45	4.41
		SIM ↑	0.83	0.77	0.84	0.81
		ACC (%) ↑	76.62	65.62	90.15	76.95
插入	open	no-edit WER (%) ↓	4.75	18.39	2.71	16.16

表 8: Ming-Freeform-Audio-Edit 上的声学编辑结果。较低的 WER、RDE 和 RAE，以及较高的 SIM 表示更好的性能。最佳结果以粗体显示。按照基准协议，Pitch Alteration 仅报告 WER 和 SIM。

任务	指标	Ming-UniAudio-Edit [15] (中文	英文)	FireRedAudio (中文	英文)
速度改变	WER (%) ↓	5.88	17.53	2.00	4.43
	SIM ↑	0.66	0.57	0.79	0.71
	RDE (%) ↓	6.36	5.92	2.60	4.02
音高改变	WER (%) ↓	7.45	13.37	2.00	3.04
	SIM ↑	0.36	0.24	0.52	0.44
音量改变	WER (%) ↓	1.71	1.35	1.60	1.30
	SIM ↑	0.86	0.80	0.94	0.93
	RAE (%) ↓	14.90	11.70	2.39	3.74

我们在 Ming-Freeform-Audio-Edit [15] 上评估自由形式语音编辑，不提供编辑位置或对齐信息。语义编辑涵盖 basic 和 open 设置下的删除、插入和替换，其中 open 对应原始的完整设置。声学编辑涵盖速度改变、音高改变和音量改变。

对于语义编辑，FireRedAudio 在大多数指标和设置上一致优于 Ming-UniAudio-Edit。与 Ming-UniAudio-Edit 相比，FireRedAudio 实现了显著更低的 WER，尤其是在 open 删除设置中，中文 WER 从 22.92% 降至 10.49%，英文从 27.60% 降至 16.65%。它还获得了更高的 ACC 分数；在 open 替换设置中，中文 ACC 从 76.62% 提高到 90.15%。两种模型的 SIM 分数通常可比，FireRedAudio 在几个设置中略有优势。对于声学编辑，FireRedAudio 在所有三个任务中均大幅优于 Ming-UniAudio-Edit。速度改变的中文 WER 从 5.88% 降至 2.00%，英文从 17.53% 降至 4.43%，而 SIM 从 0.66/0.57 提高到 0.79/0.71。音高改变的中文 WER 从 7.45% 降至 2.00%，英文从 13.37% 降至 3.04%，SIM 从 0.36/0.24 提高到 0.52/0.44。音量改变在两种语言中均实现了超过 0.90 的 SIM，RAE 从 14.90/11.70 大幅降至 2.39/3.74。这些结果表明，解耦的输入表示和渐进式多任务训练使 FireRedAudio 在自由形式语义和声学编辑中均达到强性能。

4.5 具有秒级时间戳精度的结构化长格式音频组织

图 3: 四种代表性的长格式音频理解能力：结构化组织、长格式摘要、时间与内容之间的双向检索，以及跨录音分布证据的全局分析。我们对时间定位的定量评估聚焦于结构化组织。

FireRedAudio 接受长达一小时的录音，并支持图 3 所示的四种代表性长格式音频理解能力。我们仅对结构化组织进行时间定位的定量评估，其中模型将录音转换为包含时间间隔和对应内容的结构化条目。该输出直接暴露了预测的时间边界是否与录音内容一致。

为量化结构化组织中的时间定位，我们评估了 50 个以语音为主的录音，在 5、10、20、30 和 50 分钟时长组中各包含 10 个录音。每个预测条目包含一个区间 $[s, e]$ 和对应的语音内容。由于有效的结构化输出可能以不同方式分割同一录音，我们直接对每个预测条目进行评分，而非将其与固定的参考分割进行匹配。一个音频裁判判断预测区间是否包含该条目指定的语音内容；说话人、情感和其他非时间字段被排除。Qwen3.5-Omni-Plus [7] 作为音频裁判。两个系统的输出均使用相同的裁判和提示进行评估。在 strict@0 下，仅当精确区间包含完整的指定语音内容、不包含额外语音且前后静音不超过 0.3 秒时才算通过。在 content@ δ 下，区间被扩展为 $[s - \delta, e + \delta]$ ，如果扩展后的音频段中完整包含指定的语音内容则算通

过。我们报告 $\delta = 0.5$ 和 1.0 秒。更高的通过率表示更好的时间定位。Overall 分数是在 50 个录音的所有提取片段上计算的，而不是对五个时长组的分数进行平均。

表 9: 结构化长格式音频组织的无参考边界时间定位通过率 (%)，按时长分组。每组包含 10 个以语音为主的录音。Qwen3.5-Omni-Plus 作为音频裁判；越高越好。最佳结果以粗体显示。

时长	strict@0	content@0.5	content@1.0	:--	:--	:--	:--	:--	:--	:--		FireRedAudio	Qwen3.5-Omni-Plus	FireRedAudio	
Qwen3.5-Omni-Plus	FireRedAudio	Qwen3.5-Omni-Plus		5 分钟	76.9	45.9	100.0	98.1	100.0	99.3		10 分钟	68.9	53.9	99.0
99.4	99.0		20 分钟	73.8	61.0	96.8	95.6	98.0	98.4		30 分钟	77.7	71.1	98.4	98.2
98.7	98.4		50 分钟	72.5	47.5	93.9	85.5	94.4	92.9		Overall	73.6	56.6	96.1	92.6
96.7	96.5														

表 9 显示 FireRedAudio 在 5 至 50 分钟录音的结构化组织中保持了准确的时间定位，并在所有三个总体指标上优于 Qwen3.5-Omni-Plus。在 strict@0 下，它领先 17.0 个百分点，表明更精确和紧凑的区间放置。在 0.5 秒边界容差下，其总体通过率超过 96%，并且在每个时长组（包括 50 分钟录音）中均保持在 93% 以上。高 content@1 通过率进一步表明，在预测区间两端各扩展一秒后，大多数结构化条目都包含其对应的语音内容。以 content@1 作为秒级时间戳精度的标准，FireRedAudio 在 5 至 50 分钟的录音上保持了高时间定位性能。这种鲁棒性可能得益于我们的长格式数据构建，该方法将准确标注的块合并为更长的录音，将块相对时间戳转换为录音级时间戳，并协调块级说话人标签为全局一致的说话人身份。时间戳转换直接监督时间-内容对齐，而说话人标签协调促进跨块说话人一致性。

5 结论

我们提出了 FireRedAudio，一个通用的音频语言模型，在共享的 9B 参数 LLM 中统一了 ASR、音频理解、零样本 TTS、指令 TTS 以及语义和声学语音编辑。其核心设计根据每个音频片段的功能解耦连续输入表示：专用的音频编码器服务于理解，而可重建的 RedAE 路径服务于语音生成。结合流匹配 DiT 和渐进式多任务训练，此设计使得长达一小时的录音音频理解成为可能，并在评估任务中取得了有竞争力或领先的性能。对于长格式音频的结构化组织，FireRedAudio 展示了秒级时间戳精度。总体而言，这些结果支持输入侧表示解耦作为一种可行方式，以适应音频理解和语音生成的不同要求，同时保留共享的语言和推理主干。

作者贡献

作者在各组内按名字字母顺序排列。

核心贡献者：Feiyu Shen, Fenglong Xie, Junjie Li, Kun Xie, Lei Xie, Xu Tang, Xuelong Geng, Yan Jia, Yao Hu, Yichen Han, Yichen Wu, and Ziqi Dai。

贡献者：Junjie Chen, Kai Huang, Manzhen Wei, and Yixuan Li。